Connecting a Backend to Postgres

Overview

This practical exercise will guide you through connecting a Node.js backend application to a PostgreSQL database without using an ORM. The focus is on understanding the fundamentals of database connectivity.

Prerequisites

- Postgres installed

Part 1: Database Setup

1.1 Start PostgreSQL Service

macOS (Homebrew)

```
# Check PostgreSQL status
pg_isready

# If using Homebrew, check services
brew services list

# Start PostgreSQL if not running
brew services start postgresql
```

1.2 Access PostgresSQL and Create the Database

macOS (Command Line)

Log in to PostgreSQL:

```
# If you're using your own user account
psql
```

Windows (Using PSQL Shell)

Open SQL Shell (psql)

1. **Create a database and table:**

```
CREATE DATABASE student_records;
\c student_records
```

2. **Add some sample data:**

```
INSERT INTO students (name, email, course, enrollment_date)
VALUES
  ('Sonam Dorji', 'sonam@example.com', 'Web Development', '2023-01-15'),
  ('Tashi Dema', 'tashi@example.com', 'Data Science', '2023-02-10'),
  ('Sonam Dema', 'dema@example.com', 'Mobile App Development', '2023-01-20');
  ('Karma Dorji', 'karma@example.com', 'Mobile App Development', '2023-01-20');
```

3. **Verify the data:**

```
SELECT * FROM students;
```

4. **Exit PostgresSQL**

```
\q
```

Part 2: Backend Setup

2.1 Create a Backend Project

1. **Create a project folder and initialize it:**

```
mkdir db-connection

cd db-connection

npm init -y
```

2. **Install required packages**:

```
npm install express pg cors
```

2.2 Create a Basic Database Test Script

Create a file named db-test.js to test your database connection:

```
const { Pool } = require('pg');

// Create a connection pool - ADJUST YOUR CREDENTIALS AS NEEDED
const pool = new Pool({
  user: 'postgres',      // Default PostgreSQL user (adjust if different)
  host: 'localhost',
  database: 'student_records',
  password: '',          // Add your password if you've set one
  port: 5432             // Default PostgreSQL port
});

// Test the connection and run a query
async function testConnection() {
  let client;

  try {
    // Get a client from the pool
    client = await pool.connect();
    console.log('Connected to PostgreSQL database!');

    // Run a simple query
    const result = await client.query('SELECT * FROM students');

    // Print the results
    console.log('Students in database:');
    console.table(result.rows);

    // Count rows
    console.log(`Total students: ${result.rowCount}`);
  } catch (err) {
    console.error('Database connection error:', err);
  } finally {
    // Release the client back to the pool
    if (client) client.release();

    // Close the pool
    await pool.end();
    console.log('Connection pool closed');
  }
}

// Run the test
testConnection();
```

Run this script to test your connection:

```
node db-test.js
```

2.3 Create the Server

Now create a file named server.js for your API:

```
const express = require('express');
const { Pool } = require('pg');
const cors = require('cors');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Create PostgreSQL connection pool - ADJUST YOUR CREDENTIALS AS NEEDED
const pool = new Pool({
  user: 'postgres',      // Default PostgreSQL user (adjust if different)
  host: 'localhost',
  database: 'student_records',
  password: '',          // Add your password if you've set one (required on Windows)
  port: 5432             // Default PostgreSQL port
});

// Test database connection on startup
pool.connect()
  .then(client => {
    console.log('Connected to PostgreSQL database');
    client.release();
  })
  .catch(err => {
    console.error('PostgreSQL connection error:', err);
  });

// Root endpoint
app.get('/', (req, res) => {
  res.send('Student Records API is running (PostgreSQL)');
});

// GET API: Fetch all students
app.get('/api/students', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM students');
    res.json(result.rows);
  } catch (err) {
    console.error('Error fetching students:', err);
    res.status(500).json({ error: 'Failed to fetch students' });
  }
});
```

```
// POST API: Add a new student
app.post('/api/students', async (req, res) => {
  const { name, email, course, enrollment_date } = req.body;

  // Validate input
  if (!name || !email || !course || !enrollment_date) {
    return res.status(400).json({ error: 'All fields are required' });
  }

  try {
    const result = await pool.query(
      'INSERT INTO students (name, email, course, enrollment_date) VALUES ($1, $2, $3, $4) RETURNING *',
      [name, email, course, enrollment_date]
    );

    res.status(201).json(result.rows[0]);
  } catch (err) {
    console.error('Error adding student:', err);

    // Handle duplicate email error
    if (err.code === '23505') { // PostgreSQL unique violation error code
      return res.status(409).json({ error: 'Email already exists' });
    }

    res.status(500).json({ error: 'Failed to add student' });
  }
});

// Start server
const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Start the server:

```
node server.js
```

Part 3: Testing the API

3.1 Test using a Web Browser

1. Open your browser and navigate to:

   `http://localhost:5000/api/students`

3.2 Test using cURL

In a separate terminal:

```
# Fetch all students
curl http://localhost:5000/api/students
```

```
# Add a new student
curl -X POST http://localhost:5000/api/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Karma Pema","email":"karma@example.com","course":"Cloud
Computing","enrollment_date":"2023-03-15"}'
```